

Problem

Find the values of unknown variables in the given parametric equation of a curve :

$$x = \left(t * \cos(\theta) - e^{M|t|} \cdot \sin(0.3t) \sin(\theta) + X \right)$$

$$y = \left(42 + t * \sin(\theta) + e^{M|t|} \cdot \sin(0.3t) \cos(\theta) \right)$$

1. Initial Understanding and Thought Process

When I first read, my intuition was to relate it to a machine learning parameter estimation problem, similar to Linear Regression.

In Linear Regression, we assume that the data follows a model

$$y = mx + c$$

where the parameters m, c are unknown and we need to find a best fit line (find m, b) that minimizes the loss between predicted and actual values

Eventhough the given problem is significantly different, I noticed a similar underlying idea.

Instead of fitting a straight line, here we are fitting a parametric curve defined by the equations

$$x = f(t, \theta, M, X) \text{ and } y = g(t, \theta, M, X)$$

The unknown parameters are

- θ
- M
- X

Unlike Linear Regression, these parameters appear inside (sin,cos and e), making this a **nonlinear parameter estimation** problem rather than a linear one.

The given file contains only sampled (x, y) coordinates of the curve. Therefore, my task is to recover the values of the unknown parameters that could have generated these observations.

My understanding of the problem:

- I initially wondered whether t was another unknown parameter. But after examining the equations, I realized that t **functions** like a timestamp while tracing a path.
- t (from 6 to 60) has been plugged into the equation. Each value of t produces one point (x,y) on the curve. So it is not estimated, rather it is simply varied over the given interval to generate the complete curve. That brings to my second understanding:
- For any candidate values of $\{\theta, M, \text{ and } X\}$, I can generate an entire curve' by evaluating the equations over uniformly sampled values of (t).
- The generated curve can then be compared against the observed points.
- The **quality** is determined by how closely the predicted curve matches the observed curve.
- the judging criteria specifies the L1 distance between uniformly sampled points, I concluded that this is an optimization objective.

This led me to formulate the problem as an **inverse parameter estimation** task:

1. Start with some initial values for (θ) , (M), and (X).
2. Generate the predicted curve using the given equations.
3. Compare the predicted points with the observed points using the L1 distance.
4. ^Adjust the parameters to reduce the error.
5. Repeat until the error converges or reaches a minimum.

^more like : use an optimization algorithm to adjust the parameters

2. Analysing the equation

Problem

Find the values of unknown variables in the given parametric equation of a curve :

$$x = \left(t * \cos(\theta) - e^{M|t|} \cdot \sin(0.3t) \sin(\theta) + X \right)$$

$$y = \left(42 + t * \sin(\theta) + e^{M|t|} \cdot \sin(0.3t) \cos(\theta) \right)$$

From the equation it is evident that the shape and scale predominantly depends on M more than theta, because at M = 0, the linear term t*cos(theta) wins (since theta is constant it is linear)

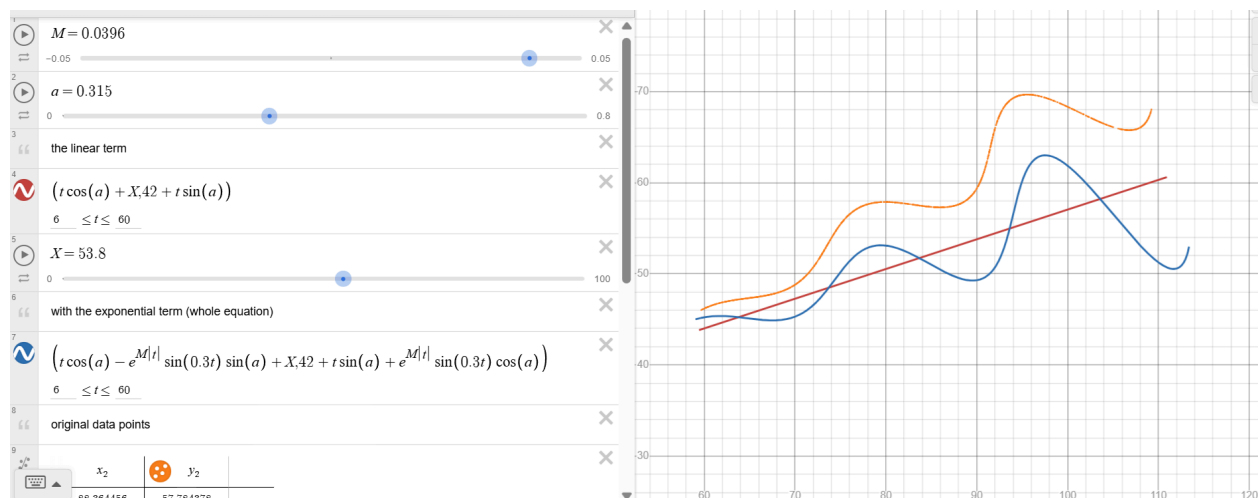
t*(cosθ, sinθ) - trend scaled by t

Wiggle : the exponential part

X,42 : translates the whole line

3. Desmos Implementation

In desmos:



here , yellow scatter plot represents the points in xy_data.csv

Red line represents the linear term t(cos,sin)

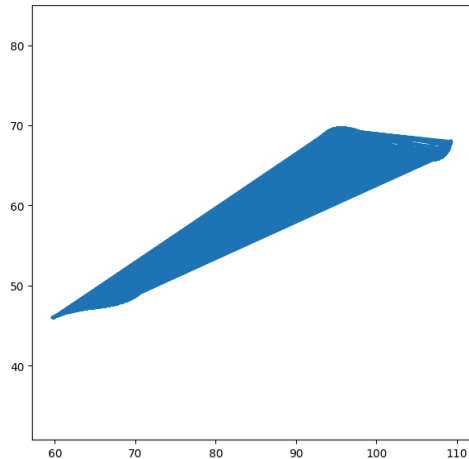
Blue line represents the original equation.

Generalizing the curve:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \underbrace{t \cdot \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}}_{\text{trend}} + \underbrace{e^{M|t|} \cdot \sin(0.3 t) \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}}_{\text{wiggles}} + \begin{bmatrix} x \\ 42 \end{bmatrix}$$

4. Analyzing the data

Mistake i made: i initially assumed the data was ordered, but while plotting I realised it was not.



(it took me good time for this, cause i thought plotting it wrong :/)

$x \approx t \cdot \cos\theta + X$ and t ranges over $(6,60)$

x is approximately an increasing function of t , then **sorting the data by x value approximately sorts it by t !**

Note to self: it'll be slightly wrong wherever the wiggle term is strong enough to make x volatile

Using the values i eye-balled from Desmos i computed the loss

After sorting i got L1 distance loss 0.32704699401586934

$\theta = 0.522$

$M = 0.03$

$X = 55$

5. Optimization Algorithms

I jumped straight to Grid search the data had only around 1.5K rows and i considered it as my coarse search

Loss from grid search L1 Loss : 0.304697

Theta : 30.000 degrees

M : 0.030000

X : 55.000

Okay so either there's

- some wrong estimation in parameters or
- t labelling is wrong for some values.

I had these constraints below in mind and researched for optimization algorithms that best fit this problem:

- ☐ Only **3 continuous variables**: M, θ, X
- ☐ **Bounded search space**
- ☐ **Nonlinear** (sin, cos, exponential)
- ☐ **L1 loss**: which is **non-differentiable**
- ☐ **unknown correlation** between (x, y) and the parameter t

I chose DE because it works with all the constraints i have for this problem.

3 continuous variables	DE can be used on optimization problems that are not continuous [4]
------------------------------	---

Bounded parameters	The function handles Bounds for variables. [3]
-----------------------	--

Nonlinear equations	Finds the global minimum of a multivariate function. [3]
------------------------	--

L1 (non-differentia ble) loss	does not use gradient methods to find the minimum. [3]
-------------------------------------	--

Since the correlation between the observed points and the parameter t was unknown,

I used a nearest-neighbor L1 loss instead of a point-wise L1 loss.

Because it would make the optimization independent of the ordering of the given data.

Therefore, DE is suitable when/for:

- ☒ Only ~~3 continuous variables: M, θ, X~~
- ☒ **Bounded search space**
- ☒ **Nonlinear** (~~sin, cos, exponential~~)
- ☒ **L1 loss**: which is **non-differentiable**
- ☒ **unknown correlation** between (x,y) and the parameter t

For DE:

```
Theta : 29.999805°  
Theta : 0.523595 rad  
M      : 0.03000055  
X      : 54.998766  
Loss   : 0.02672840
```

For this assignment i concluded this as my final values.

(there's still room for exploration but due to assignment constraints and rules i chose this to be my final value)

6.Final DESMOS CODE:

```
"  
\left(t\cos\left(0.523595372205\right)-e^{\{0.03000055\left|t\right|\}}\sin\left(0.3t\right)\sin\left(0.523595372205\right)+54.998766,42+t\sin\left(0.523595372205\right)+e^{\{0.03000055\left|t\right|\}}\sin\left(0.3t\right)\cos\left(0.523595372205\right)\right)  
"
```

7. References:

1. "tour-of-optimization-algorithms," available at:
<https://machinelearningmastery.com/tour-of-optimization-algorithms/>
2. "DE-medium," available at:
https://medium.com/@reshma_shaji/differential-evolution-what-it-is-and-how-does-it-work-81d3415c2367

3. "Pyswarms-documentation," available at:
https://pyswarms.readthedocs.io/en/latest/examples/tutorials/basic_optimization.html#Optimizing-a-function-with-bounds
4. "Differential Evolution-scipy-docs," available at:
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html#scipy.optimize.differential_evolution
5. "DE-wikipedia," available at: https://en.wikipedia.org/wiki/Differential_evolution